

Smart Loaders Documentation

1 Introduction

The `smart_loaders.py` module provides input abstraction for CTCC: it chooses `yaml_loaders` or `json_loaders` at import time based on the `CTCC_INPUT_MODE` environment variable, then re-exports all `load_*` and related symbols from the chosen backend. Calculation scripts import from `smart_loaders` `import load_project_technical_details, load_financing_details, etc.`, and get the same API regardless of whether input is YAML or JSON; no code changes are needed when switching mode.

2 Method Overview

2.1 Purpose

1. At import time: read `CTCC_INPUT_MODE` (default `yaml`); if `json`, from `json_loaders import *`; else from `yaml_loaders import *`.
2. Expose `get_input_mode()` so callers can query the current mode ("`json`" or "`yaml`").
3. Provide mode-aware raw-data helpers: `get_project_data_raw()`, `get_financing_data_raw()`, `get_physical_data_raw()` (return the raw dict for project technical details, financing, or physical details from either YAML or JSON).
4. Provide `load_terrain_miles()` that uses `get_physical_data_raw()` and returns `terrain["terrain_miles"]`.
5. Re-export all `load_*` and other symbols from the active backend so scripts use a single import.

2.2 Calculation Approach

Dispatch only; no cost/benefit formulas. The module does not implement load logic; it only selects the backend and re-exports. All `load_*` implementations live in `yaml_loaders.py` or `json_loaders.py` (see their documentation).

2.3 Inputs and Outputs

Inputs:

- CTCC_INPUT_MODE environment variable (read once at import): `yaml` (default) or `json`.

Outputs:

- Same `load_*` API and return types as `yaml_loaders` or `json_loaders`; callers receive dataclasses or dicts from the chosen backend.

2.4 Integration with Other Scripts

- Every calculation script that loads configuration imports from `smart_loaders` (e.g. `from smart_loaders import load_project_technical_details, load_financing_details`).
- `smart_loaders` imports either `json_loaders` or `yaml_loaders` at import time; it does not call both in the same run.

3 Key Functions

3.1 Module-level logic (conditional import)

At import time:

```
1 _input_mode = os.environ.get('CTCC_INPUT_MODE', 'yaml').lower()
2 if _input_mode == 'json':
3     from json_loaders import *
4 else:
5     from yaml_loaders import *
```

All `load_*` and other public symbols from the selected module are then available as `smart_loaders.load_pr` etc.

3.2 `get_input_mode()`

Returns the current input mode: `"json"` or `"yaml"` (the value determined at import time).

3.3 `get_project_data_raw()`

Returns the raw dict for project technical details (keys `project`, `timeline`). In JSON mode calls `json_loaders.data_source.get_data("01_project_technical_details")`; in YAML mode loads `YAMLS_DIR / 01_project_technical_details.yaml`.

3.4 `get_financing_data_raw()`

Returns the raw financing dict. In JSON mode `get_data("03_financing")`; in YAML mode loads `03_financing.yaml`.

3.5 `get_physical_data_raw()`

Returns the raw physical details dict (includes `terrain` with `terrain_miles`, etc.). In JSON mode `get_data("02_project_physical_details")`; in YAML mode loads `02_project_physical_details`

3.6 `load_terrain_miles()`

Calls `get_physical_data_raw()`, then returns `physical_details["terrain"]["terrain_miles"]` (dict mapping terrain type to miles). Raises `KeyError` if `terrain` or `terrain_miles` is missing.

3.7 Re-exported `load_*` (see `yaml/json loader docs`)

All other `load_*` functions (e.g. `load_project_technical_details`, `load_financing_details`, `load_congestion_curtailement_reductions`, `load_physical_details`, `load_primary_bcr_config`) are defined in `yaml_loaders.py` or `json_loaders.py` and re-exported here; their parameters, return values, and key logic are documented in those modules.

4 Example

Script uses the same imports regardless of mode:

```
1 from smart_loaders import load_project_technical_details,
   load_financing_details
2 details = load_project_technical_details()
3 financing = load_financing_details()
```

To force JSON mode (e.g. before importing):

```
1 import os
2 os.environ["CTCC_INPUT_MODE"] = "json"
3 # Then import smart_loaders (or run script with CTCC_INPUT_MODE=json
   set in shell)
4 from smart_loaders import load_project_technical_details
5 details = load_project_technical_details() # data from JSON
```